Da Semana 10 à Semana 11

Conectando o CSS avançado, o DOM e os formulários ao HTML semântico, localStorage, fetch e Browser APIs

Desenvolvimento Front-End [React] T4 | Módulo 1 | Este material não traz conteúdo novo — ele mostra, ponto a ponto, como o que você já praticou nas semanas anteriores (CSS e Flexbox, DOM com JavaScript, formulários, `async/await` e CSS avançado da S10) é a **base literal** do que vamos fazer na Semana 11. A ideia é simples: **nada aqui é assunto desconhecido, é o mesmo conhecimento crescendo.**

Como ler este material

Cada seção segue o mesmo formato: primeiro, **o que você já sabe** (com o código como ele apareceu nas semanas anteriores); depois, **o que a Semana 11 acrescenta**; por fim, uma comparação lado a lado entre a versão "de antes" e a "de agora" do mesmo problema.

> **O fio condutor: um projeto novo que usa tudo o que você já sabe**
>
> Nas semanas anteriores você evoluiu a página da Bikcraft. Agora a turma muda de produto e de cliente: o **PrevisaoCerta**, painel climático da transportadora **AgroLog**. O problema-âncora é real: *os despachantes precisam acompanhar o tempo nas cidades de coleta e entrega — a ferramenta tem que ser acessível, lembrar das rotas e mostrar dados reais.* A tabela mostra a linha do tempo.

Semana	O que você construiu	Ferramenta usada
06	Código assíncrono: esperar uma resposta sem travar a página	`async` / `await`
08	JavaScript mexendo na página	DOM: `getElementById`, `createElement`, `classList`, `textContent`
09	Formulário completo, captura no `submit`, confirmação na tela	Formulários HTML + JavaScript
10	Layout responsivo, Flexbox, especificidade, `:focus-visible`, variáveis CSS, Grid sem media query	CSS avançado (e bibliotecas, em teoria)
11	Um painel acessível, com memória e dados reais da internet	**Assunto desta semana**

CONEXÃO 1 — Do formulário e do foco à acessibilidade de verdade

O que você já sabe (Semanas 09 e 10)

Você já monta um formulário HTML, captura os dados no `submit` e já usou, na paginação da S10, `aria-current="page"` e `:focus-visible` no CSS para o foco por teclado. Ou seja: **você já fez acessibilidade sem dar esse nome**.

```
<form id="form-contato">
  <label for="nome">Nome</label>
  <input type="text" id="nome" name="nome" />
  <button type="submit">Enviar</button>
</form>

a:focus-visible { outline: 2px solid var(--destaque); }
```

O que a Semana 11 acrescenta

- **Landmarks e hierarquia:** `<header>`, `<nav>`, `<main>`, `<footer>`, um só `<h1>` — a página inteira, não só o formulário, passa a ser compreensível pela máquina.
- **Medir com o Lighthouse:** uma nota objetiva de Acessibilidade e SEO *antes* e *depois*.
- **ARIA e teclado:** `aria-label`, `aria-live`, skip link, `alt` decorativo × informativo.
- **SEO on-page:** `<title>` e `<meta name="description">`.

Comparando o código: o mesmo formulário, antes e depois

SEMANA 09/10 — funciona e é bonito

```
<div class="busca">
  Cidade
  <input type="text">
  <div class="btn">Buscar</div>
</div>
```

SEMANA 11 — a máquina entende

```
<form id="form-busca" role="search">
  <label for="input-cidade">Nome da
cidade</label>
  <input type="search" id="input-cidade" />
  <button type="submit">Buscar</button>
</form>
```

> **O que realmente mudou?**
>
> O visual pode ser o mesmo. O que muda é que cada peça agora **diz o que é**: o leitor de tela anuncia "Nome da cidade, busca" e o botão é um botão de verdade (foco por Tab, Enter e Espaço de graça).

CONEXÃO 2 — Do DOM e dos arrays à memória do navegador

O que você já sabe (Semana 08)

Você já cria elementos com `createElement`, usa `classList` e `textContent`, seleciona com `getElementById` / `querySelector` e percorre listas com `forEach` e `filter`. A lista que você desenha na tela vem de um **array na memória do JavaScript**:

```
const cidades = ['Florianópolis', 'Curitiba'];

function renderizarLista() {
  lista.innerHTML = '';
  cidades.forEach(function (nome) {
    const item = document.createElement('li');
    item.textContent = nome;
    lista.appendChild(item);
  });
}
```

O que a Semana 11 acrescenta

- **Persistência:** a mesma lista passa a viver no `localStorage` — sobrevive ao F5 e ao fechar a aba.
- **JSON:** como o `localStorage` só guarda texto, o array é convertido com `JSON.stringify` e reconstruído com `JSON.parse`.
- **O `null` da primeira visita:** quando nada foi salvo, o `getItem` devolve `null` — por isso o `|| []`.
- **Timers:** `setTimeout` para um aviso que some sozinho.

Comparando o código: a mesma lista, agora com memória

SEMANA 08 — some no F5

```
const cidades = [];            // memória da
página

function adicionar(nome) {
  cidades.push(nome);
  renderizarLista();
}
```

SEMANA 11 — sobrevive ao F5

```
function carregarCidades() {
  return JSON.parse(

localStorage.getItem('previsaocerta:cidades')
  ) || [];
}
function adicionar(nome) {
  const cidades = carregarCidades();
  cidades.push({ nome: nome });
  localStorage.setItem('previsaocerta:cidades',
    JSON.stringify(cidades));
  renderizarLista();
}
```

O que realmente mudou?

> `renderizarLista()` continua exatamente igual. A única diferença é **de onde** vem o array: da variável (que zera) ou do `localStorage` (que lembra).

CONEXÃO 3 — Do async/await e dos eventos aos dados da internet e do relógio

O que você já sabe (Semanas 06 e 08/09)

Você já sabe que algumas coisas **demoram** e que o `await` espera sem travar a página. Também já escuta eventos (`click`, `submit`) e organiza o código em funções pequenas.

```
async function esperar() {
  const resultado = await algumaPromessaQueDemora();
  console.log(resultado);
}
form.addEventListener('submit', function (evento) {
  evento.preventDefault();
});
```

O que a Semana 11 acrescenta

- `fetch`: a "promessa que demora" mais comum do dia a dia — pedir dados a um servidor. **Dois** `await`: um para a resposta e outro para o `.json()`.
- **O caminho triste:** `try/catch` para a rede que cai e `response.ok` para o 404 que finge sucesso. Os **3 estados**: carregando, sucesso e erro.
- **Timers:** `setTimeout` (uma vez, depois de N ms) e `setInterval` (repete a cada N ms; guarde o id e use `clearInterval`).
- **Geolocation:** outra API do navegador, que depende de **permissão** do usuário — e o app precisa continuar útil se ele recusar.

Comparando o código: da promessa genérica ao fetch real

SEMANA 06 — conceito

```
const resultado =
  await algumaPromessa();
console.log(resultado);
```

SEMANA 11 — API real, com erro tratado

```
try {
  const resposta = await fetch(url);
  if (!resposta.ok) {
    throw new Error('Status ' +
resposta.status);
  }
  const dados = await resposta.json();
  renderizarCard(dados);
} catch (erro) {
  mostrarStatus('Não foi possível carregar.',
'erro');
}
```

O que realmente mudou?

> O `await` é o mesmo. A novidade é **o que esperar** (um servidor de verdade) e a obrigação profissional de **cobrir o que dá errado**.

CONEXÃO 4 — Do CSS avançado da S10 à identidade visual do projeto

O que você já sabe (Semana 10)

Você já usa **variáveis CSS** para guardar a cor da marca num lugar só, **mobile-first** com `min-width`, Grid com `repeat(auto-fit, minmax(...))` sem media query, `:focus-visible` e a media query `prefers-reduced-motion`.

O que a Semana 11 acrescenta

Todo esse CSS volta no PrevisaoCerta, agora com uma **identidade visual** definida no documento funcional (paleta verde/âmbar da AgroLog, contraste calculado, foco âmbar) e com os estados de interface (carregando, sucesso, erro) pintados por classes.

Aviso/recurso da S10	Como volta na S11
Variáveis CSS (`--destaque`)	Tokens da marca: `--verde-campo` , `--verde-folha` , `--ambar-foco` …
`:focus-visible`	Anel âmbar de 3px: nunca remover o outline
Grid `auto-fit`	Lista de cidades monitoradas responsiva, sem media query
`prefers-reduced-motion`	Transições desligadas para quem pede menos movimento
Mobile-first	Cabeçalho empilhado no celular e em linha a partir de 768px

Checklist: você já está pronto para a Semana 11 se...

- ✓ Sabe montar um formulário HTML com `label`, `input` e `button` e capturar o `submit` com JavaScript.
- ✓ Sabe criar elementos com `createElement`, usar `classList` / `textContent` e percorrer arrays com `forEach` e `filter`.
- ✓ Sabe o que faz `await` dentro de uma função `async`.
- ✓ Sabe declarar variáveis CSS e usar `:focus-visible` e Grid `auto-fit`.
- ✓ Sabe abrir o DevTools (F12) e olhar as abas Elements e Console.
- ✓ Entende que objeto/array e texto são coisas diferentes (será essencial para o `JSON`).

Se todos esses pontos fazem sentido, a Semana 11 não vai apresentar nada desconhecido — ela dá nome, número e ferramenta profissional (Lighthouse, Application, Network) para instintos que você já está desenvolvendo desde as semanas anteriores.